

04_profiles

Anish

2026-04-29

1. Goal of notebook

2. Load assignments + features + transactions

```
assignments <- read_csv("../data/customer_clusters.csv",
                        show_col_types = FALSE)
customer     <- read_csv("../data/customer_features.csv",
                        show_col_types = FALSE)
tx           <- read_csv("../data/online_retail_clean.csv",
                        show_col_types = FALSE)

tx$InvoiceDate <- as_datetime(tx$InvoiceDate)

# Join cluster labels onto customer table.
cust <- customer |>
  inner_join(assignments |> select(CustomerID, cluster),
             by = "CustomerID") |>
  mutate(cluster = factor(cluster))

list(
  customers_clustered = nrow(cust),
  k                   = n_distinct(cust$cluster),
  cluster_sizes       = table(cust$cluster)
)
```

```
## $customers_clustered
## [1] 4334
##
## $k
## [1] 2
##
## $cluster_sizes
##
##      1      2
## 1834 2500
```

2a. Re-label clusters in a stable order

To keep names stable across re-runs we reorder by **median log_Monetary** so cluster A is always the highers-spending group, c the lowest.

```

order_by_value <- cust |>
  group_by(cluster) |>
  summarise(med_logM = median(log_Monetary), .groups = "drop") |>
  arrange(desc(med_logM)) |>
  mutate(label = LETTERS[seq_len(n())]) |>
  select(cluster, label)

cust <- cust |>
  left_join(order_by_value, by = "cluster") |>
  mutate(cluster = factor(label, levels = LETTERS[seq_along(unique(label))])) |>
  select(-label)

# Push the same labels onto the transaction frame for $6 / $7.
tx <- tx |>
  inner_join(cust |> select(CustomerID = CustomerID, cluster),
    by = c("CustomerID" = "CustomerID"))

table(cust$cluster)

```

```

##
##      A      B
## 1834 2500

```

3. Headline summary - who they are, what they're worth

```

total_rev <- sum(cust$Monetary)

headline <- cust |>
  group_by(cluster) |>
  summarise(
    customers      = n(),
    revenue        = sum(Monetary),
    median_spend   = median(Monetary),
    median_orders  = median(Frequency),
    median_recency_d = median(Recency),
    .groups = "drop"
  ) |>
  mutate(
    pct_customers = customers / sum(customers),
    pct_revenue   = revenue / total_rev
  ) |>
  arrange(cluster)

headline |>
  mutate(across(c(revenue, median_spend), \(x) dollar(x, prefix = "£")),
    across(c(pct_customers, pct_revenue),
      \(x) percent(x, accuracy = 0.1))) |>
  kable(caption = "Cluster headlines – share of customers vs share of revenue")

```

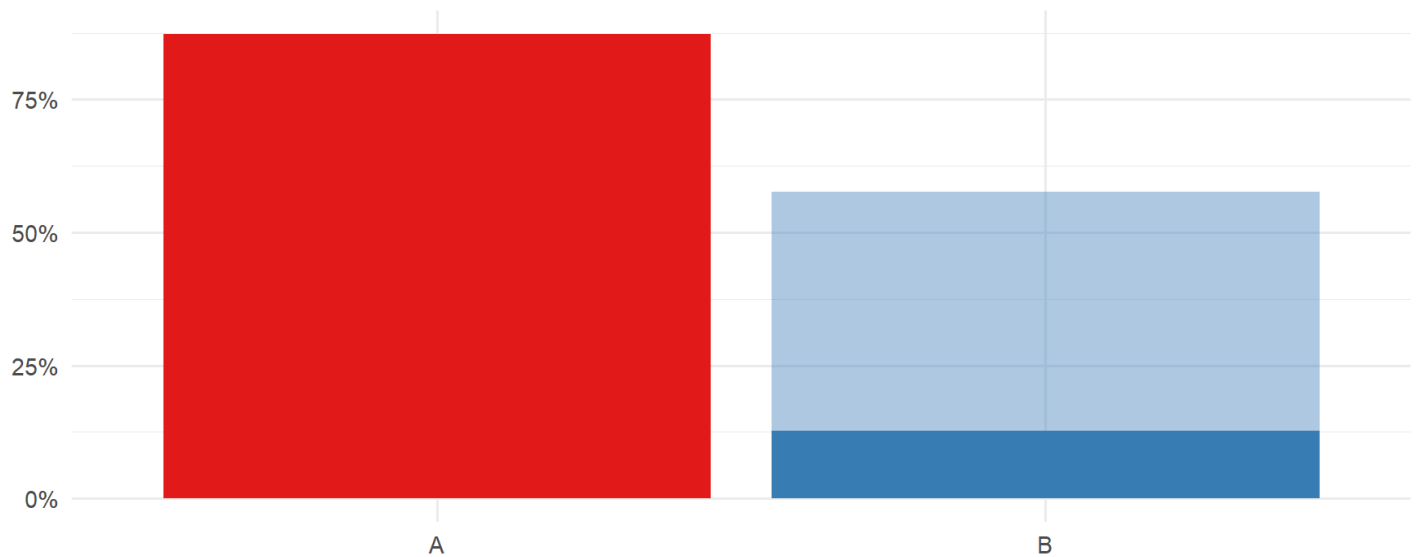
Cluster headlines — share of customers vs share of revenue

cluster	customers	revenue	median_spend	median_orders	median_recency_d	pct_customers	pct_revenue
---------	-----------	---------	--------------	---------------	------------------	---------------	-------------

cluster	customers	revenue	median_spend	median_orders	median_recency_d	pct_customers	pct_revenue
A	1834	£7,632,566	£1,871.37	5	19	42.3%	87.4%
B	2500	£1,104,662	£332.36	1	104	57.7%	12.6%

```
ggplot(headline, aes(cluster, fill = cluster)) +
  geom_col(aes(y = pct_customers), alpha = 0.4, position = "identity") +
  geom_col(aes(y = pct_revenue), alpha = 1.0, position = "identity") +
  scale_y_continuous(labels = percent_format()) +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Customers (faded) vs revenue (solid) by cluster",
       y = NULL, x = NULL) +
  theme(legend.position = "none")
```

Customers (faded) vs revenue (solid) by cluster



4. Feature profile — medians on original scales

```

profile <- cust |>
  group_by(cluster) |>
  summarise(
    n                = n(),
    Recency          = median(Recency),
    Frequency        = median(Frequency),
    Monetary         = median(Monetary),
    AvgOrderValue    = median(AvgOrderValue),
    UniqueProducts   = median(UniqueProducts),
    AvgUnitPrice     = median(AvgUnitPrice),
    TenureDays       = median(TenureDays),
    ReturnRate       = median(ReturnRate),
    .groups = "drop"
  )

profile |>
  mutate(across(c(Monetary, AvgOrderValue, AvgUnitPrice),
    \(x) dollar(x, prefix = "£", accuracy = 0.01)),
    ReturnRate = percent(ReturnRate, accuracy = 0.1)) |>
  kable(caption = "Median feature profile per cluster (original scales)")

```

Median feature profile per cluster (original scales)

cluster	n	Recency	Frequency	Monetary	AvgOrderValue	UniqueProducts	AvgUnitPrice	TenureDays	ReturnRate
A	1834	19	5	£1,871.37	£345.54	84.0	£2.77	267	10.9%
B	2500	104	1	£332.36	£229.55	19.5	£2.88	0	0.0%

5. Distributions — what each cluster looks like

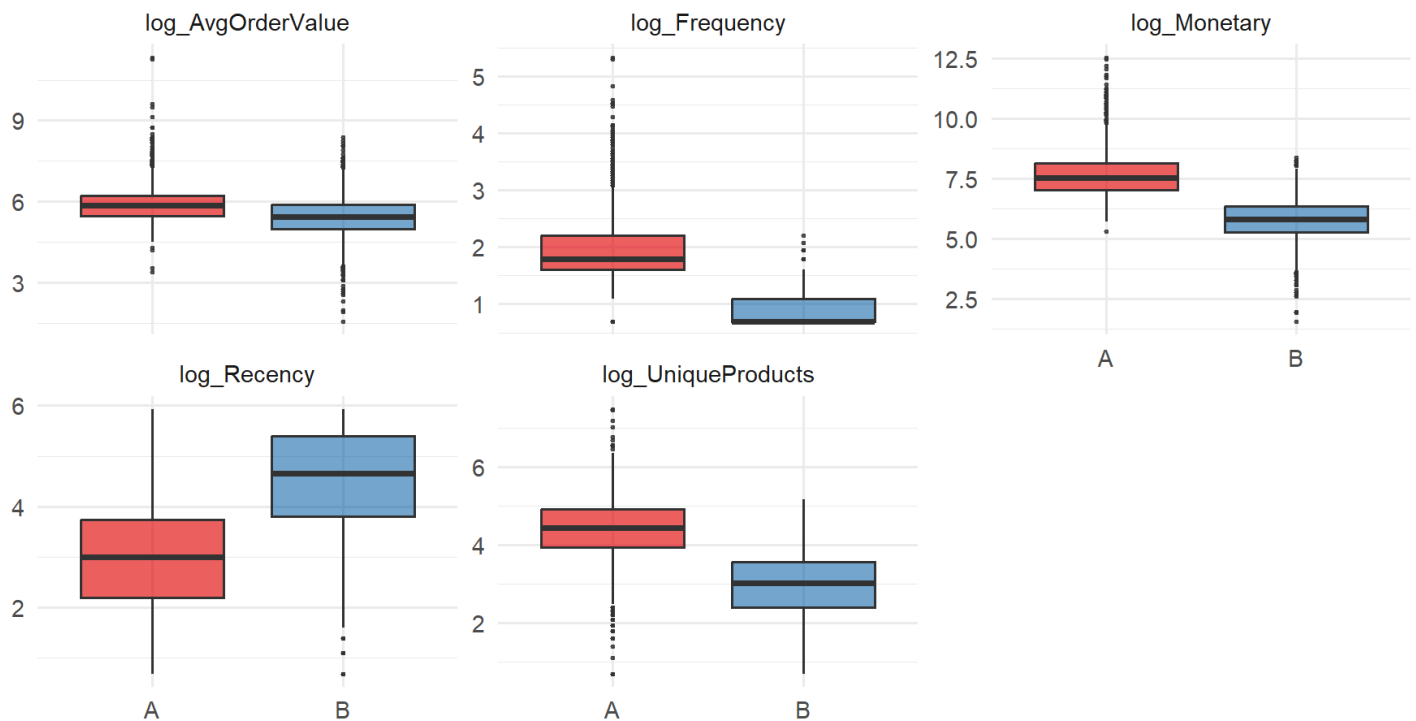
A median can mislead when distributions are skewed. Boxplots on the log scale show the spread.

```

cust |>
  select(cluster, log_Recency, log_Frequency, log_Monetary,
    log_AvgOrderValue, log_UniqueProducts) |>
  pivot_longer(-cluster, names_to = "feature", values_to = "value") |>
  ggplot(aes(cluster, value, fill = cluster)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.4) +
  facet_wrap(~ feature, scales = "free_y") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Feature distributions by cluster (log scale)",
    x = NULL, y = NULL) +
  theme(legend.position = "none")

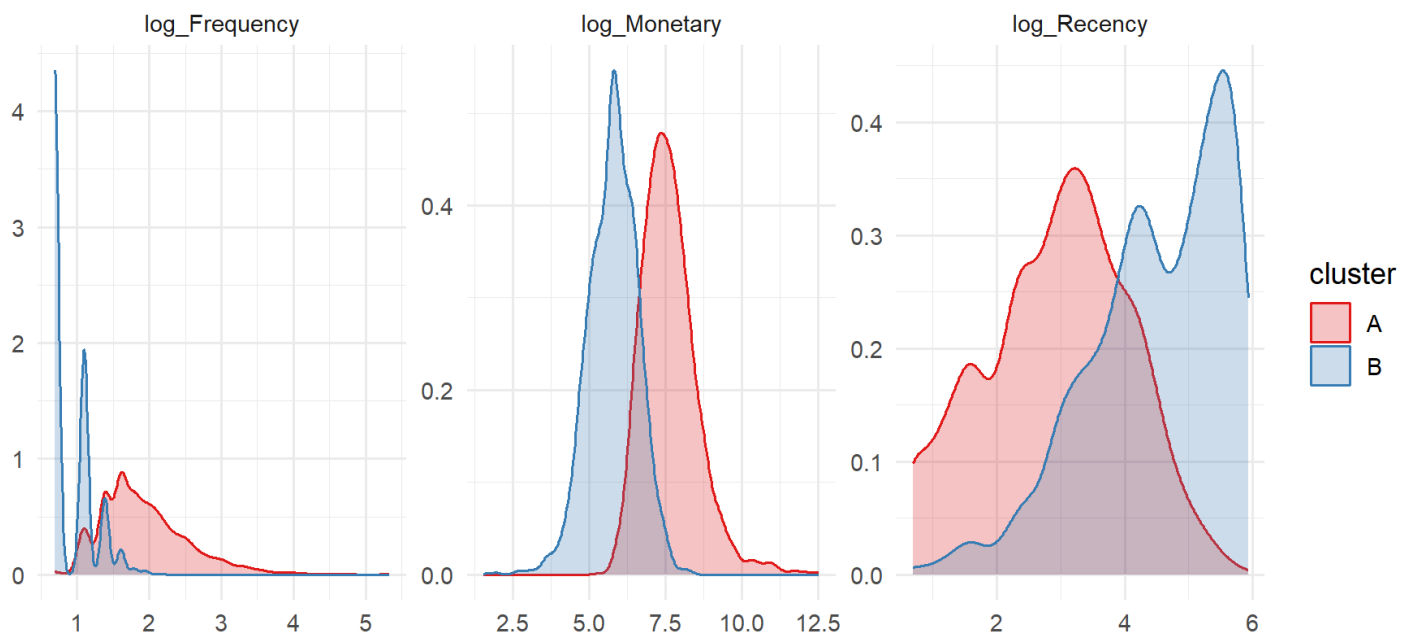
```

Feature distributions by cluster (log scale)



```
cust |>
  select(cluster, log_Recency, log_Frequency, log_Monetary) |>
  pivot_longer(-cluster, names_to = "feature", values_to = "value") |>
  ggplot(aes(value, fill = cluster, colour = cluster)) +
  geom_density(alpha = 0.25) +
  facet_wrap(~ feature, scales = "free", nrow = 1) +
  scale_fill_brewer(palette = "Set1") +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "Density of log-RFM by cluster",
       x = NULL, y = NULL)
```

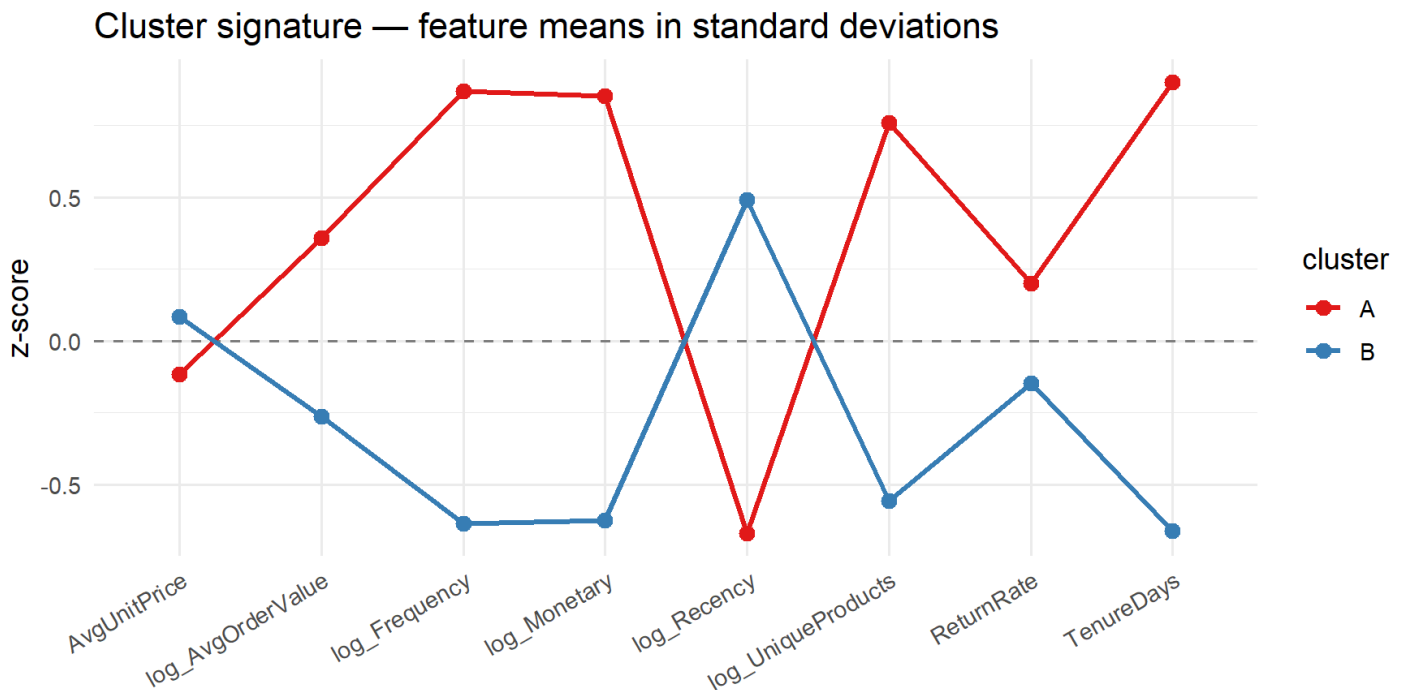
Density of log-RFM by cluster



6. Parallel-coordinate signature

```
z_profile <- cust |>
  select(cluster, log_Recency, log_Frequency, log_Monetary,
         log_AvgOrderValue, log_UniqueProducts,
         AvgUnitPrice, TenureDays, ReturnRate) |>
  mutate(across(-cluster, \(x) as.numeric(scale(x)))) |>
  group_by(cluster) |>
  summarise(across(everything(), mean), .groups = "drop") |>
  pivot_longer(-cluster, names_to = "feature", values_to = "z")

ggplot(z_profile,
       aes(feature, z, group = cluster, colour = cluster)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50") +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "Cluster signature – feature means in standard deviations",
       x = NULL, y = "z-score") +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```

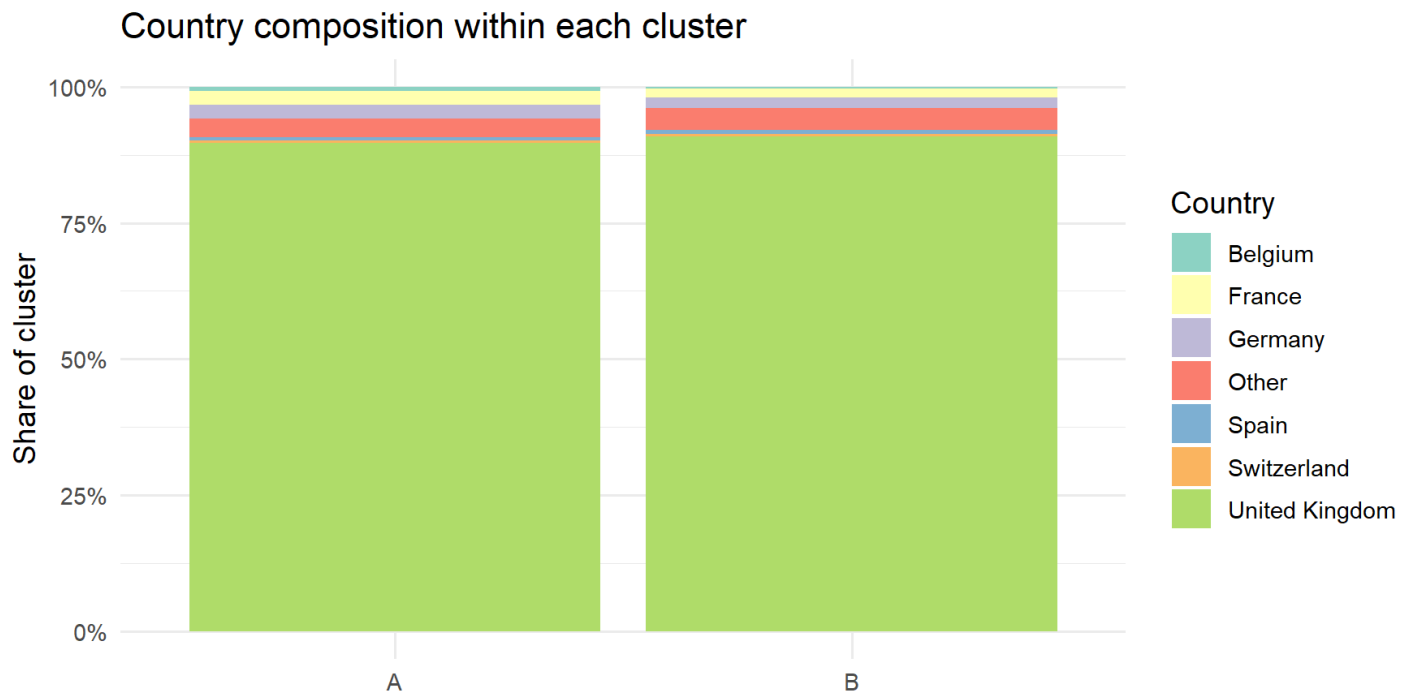


7. Geographic composition

Country was held out of the clustering on purpose (82.9% UK). Now we can ask whether cluster membership carries any country signal.

```
top_countries <- cust |> count(Country, sort = TRUE) |>
  slice_max(n, n = 6) |> pull(Country)

cust |>
  mutate(Country = ifelse(Country %in% top_countries, Country, "Other")) |>
  count(cluster, Country) |>
  group_by(cluster) |>
  mutate(share = n / sum(n)) |>
  ggplot(aes(cluster, share, fill = Country)) +
  geom_col() +
  scale_y_continuous(labels = percent_format()) +
  scale_fill_brewer(palette = "Set3") +
  labs(title = "Country composition within each cluster",
       y = "Share of cluster", x = NULL)
```



8. What does each cluster actually buy?

The profile features are aggregates; this section drops to the transaction level and asks *which products* dominate each segment.

```
top_by_cluster <- tx |>
  group_by(cluster, StockCode, Description) |>
  summarise(revenue = sum(Amount), .groups = "drop") |>
  group_by(cluster) |>
  slice_max(revenue, n = 5) |>
  ungroup()

top_by_cluster |>
  mutate(Description = str_trunc(Description, 35),
         revenue = dollar(revenue, prefix = "£")) |>
  kable(caption = "Top 5 products by revenue, per cluster")
```

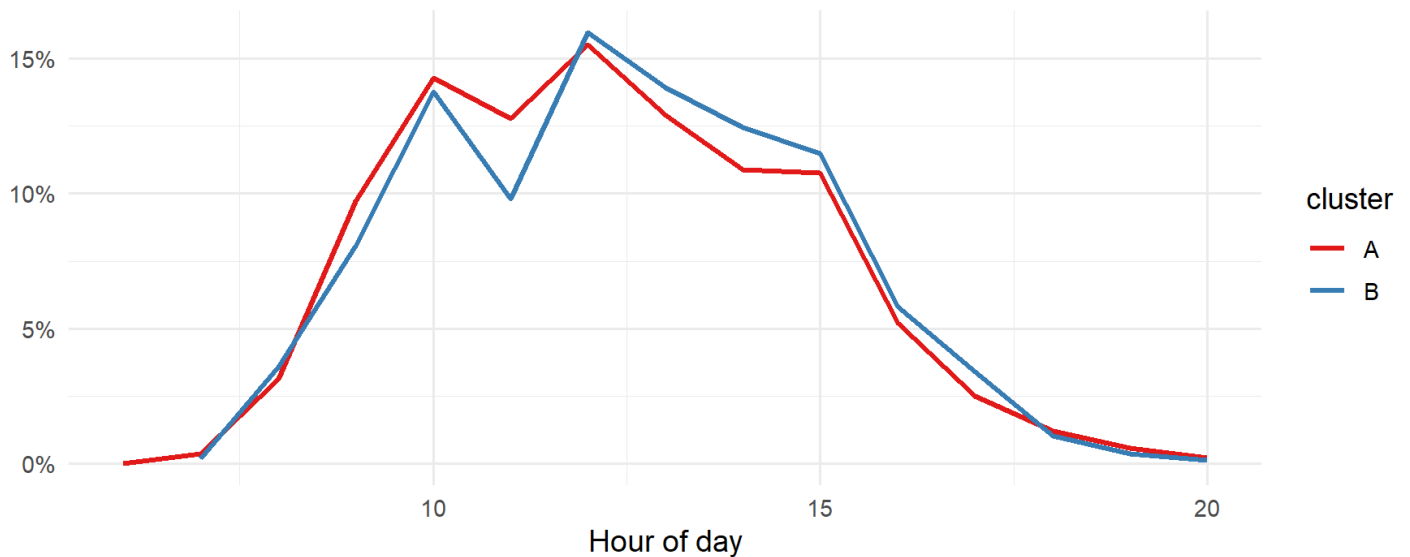
Top 5 products by revenue, per cluster

cluster	StockCode	Description	revenue
---------	-----------	-------------	---------

cluster	StockCode	Description	revenue
A	23843	PAPER CRAFT , LITTLE BIRDIE	£168,470
A	22423	REGENCY CAKESTAND 3 TIER	£126,744
A	85123A	WHITE HANGING HEART T-LIGHT HOLDER	£89,492
A	23166	MEDIUM CERAMIC TOP STORAGE JAR	£80,821
A	85099B	JUMBO BAG RED RETROSPOT	£76,620
B	22423	REGENCY CAKESTAND 3 TIER	£15,520
B	47566	PARTY BUNTING	£12,058
B	85123A	WHITE HANGING HEART T-LIGHT HOLDER	£10,900
B	85099B	JUMBO BAG RED RETROSPOT	£8,420
B	22086	PAPER CHAIN KIT 50'S CHRISTMAS	£7,211

```
tx |>
  count(cluster, Hour, wt = Amount, name = "revenue") |>
  group_by(cluster) |>
  mutate(share = revenue / sum(revenue)) |>
  ggplot(aes(Hour, share, colour = cluster)) +
  geom_line(linewidth = 1) +
  scale_y_continuous(labels = percent_format()) +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "When each cluster shops – within-cluster revenue share by hour",
       y = NULL, x = "Hour of day")
```

When each cluster shops — within-cluster revenue share by hour



9. A simple, interpretable rule for cluster membership

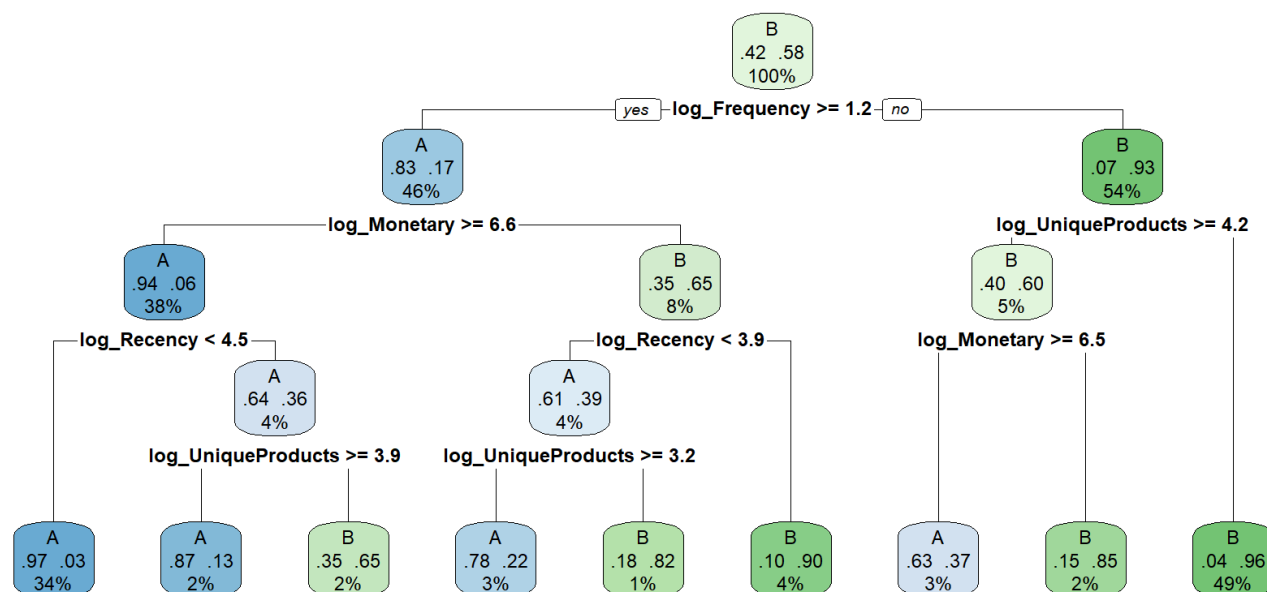

```

tree_fit <- rpart(
  cluster ~ log_Recency + log_Frequency + log_Monetary +
    log_UniqueProducts + log_AvgOrderValue,
  data = cust,
  method = "class",
  control = rpart.control(maxdepth = 4, cp = 0.005)
)

rpart.plot(tree_fit, type = 2, extra = 104, fallen.leaves = TRUE,
  main = "Decision rule approximating cluster membership")

```

Decision rule approximating cluster membership



```

pred <- predict(tree_fit, type = "class")
tab <- table(actual = cust$cluster, predicted = pred)
acc <- sum(diag(tab)) / sum(tab)

list(
  confusion_matrix = tab,
  training_accuracy = percent(acc, accuracy = 0.1)
)

```

```

## $confusion_matrix
##      predicted
## actual   A    B
##      A 1681 153
##      B 125 2375
##
## $training_accuracy
## [1] "93.6%"

```

10. Personas — naming the clusters

11. Final labeled customer table

```
final <- customer |>
  inner_join(cust |> select(CustomerID, cluster), by = "CustomerID")

write_csv(final, "../data/customer_features_labeled.csv")

list(
  rows          = nrow(final),
  cluster_sizes = table(final$cluster),
  saved_to      = "../data/customer_features_labeled.csv"
)
```

```
## $rows
## [1] 4334
##
## $cluster_sizes
##
##      A      B
## 1834 2500
##
## $saved_to
## [1] "../data/customer_features_labeled.csv"
```